Tolete, Allen Jerrome M.
CP102- M001

# Quick Links:

---

## Task 1 - Object Oriented Programming in Python

- Program Description
- Operational Logic
- Source Code
- Areas for Improvement
- Reflection

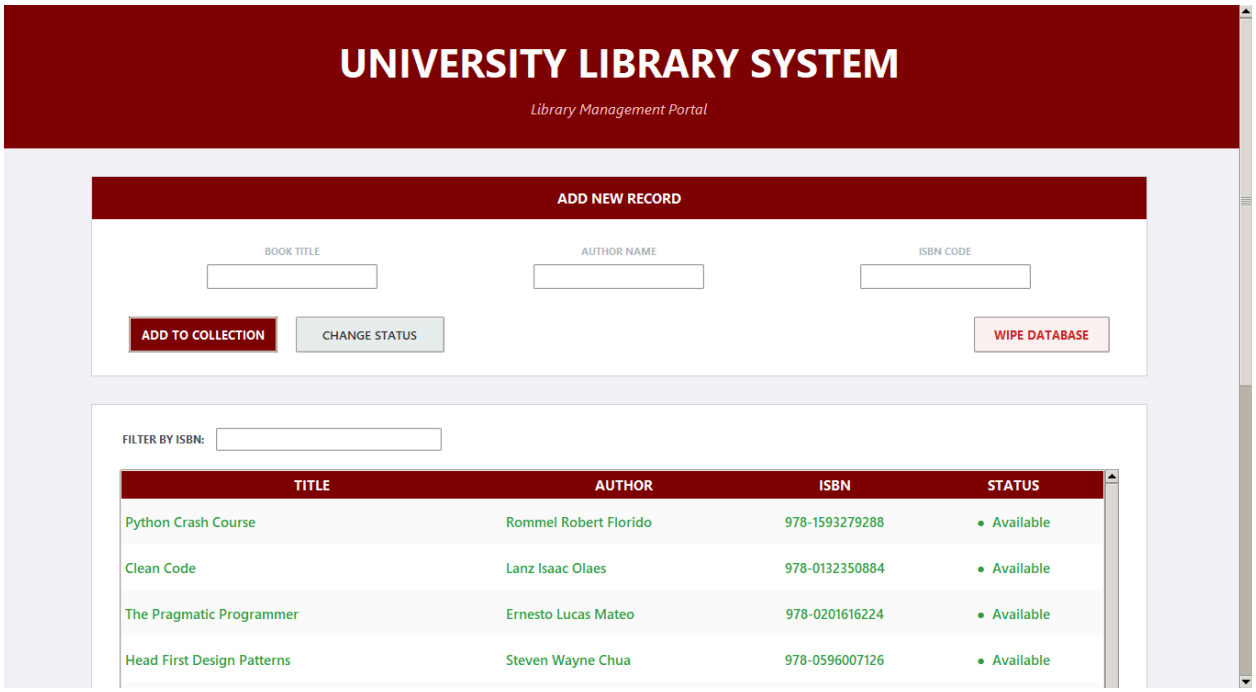
---

## Task 2 - Regular Expressions (RegEx) in Python

- Program Description
- Operational Logic
- Source Code
- Areas for Improvement
- Reflection

# Task 1 : Object Oriented Programming in Python

## Program : University Library System with UI




## I. Program Description

This program is a professional University Library Management System designed for Enverga University to streamline the tracking of book inventories and records. The system features a modern, user-friendly graphical interface built with Python's Tkinter library, allowing administrators to manage a collection of titles with specific attributes including Book Title, Author Name, and ISBN Code. The application implements a comprehensive Object-Oriented Architecture using composition, where a central Library class manages a collection of Book objects. Each Book object encapsulates its own data and availability status, while the Library class handles high-level operations such as adding new records and toggling the checkout status of items.

The program provides a clear and efficient interface divided into two primary functional areas: a compact management section and a dynamic data table. The Add New Record section, styled with a professional maroon header and white text, allows for the quick entry of new book data into the collection. Below this, the Table Card utilizes a styled Treeview to display the entire inventory, featuring green and red status indicators for "Available" and "Checked Out" books to ensure data is scannable at a glance. Users can also utilize a real-time filtering system to search the database by ISBN, maintaining an organized and responsive environment for managing university resources.

---

## II. Operational Logic

The program uses the Tkinter library for its front-end ui and a structured OOP for the back-end. It functions by utilizing 3 classes: the Book class for the data storage, the Library class to control the logic, and the Library GUI class for the visual presentation and user interaction.

# Library Imports

```
import tkinter as tk
from tkinter import messagebox, ttk
```

**import tkinter as tk**

This is the core toolkit used for creating the window and basic widgets.

- **Purpose**: It provides the foundation for the application, such as the main window (`root`), the canvas used for scrolling, and basic frames.
- **The `as tk` Alias**: This is a common convention that allows the programmer to type `tk.Label` instead of the full `tkinter.Label`, keeping the code concise.

**from tkinter import messagebox**

This is a specific sub-module within Tkinter used for pop-up dialogs.

- **Purpose**: It is used to communicate with the user outside of the main window.

- **Usage in Code**: You see this being used to show "Warnings" (like when a field is empty) or "Confirmations" (like the "Wipe all library data?" prompt).

## from tkinter import ttk

The `ttk` module stands for "Themed Tkinter".

- **Purpose**: While the standard `tk` widgets often look outdated (like Windows 95), `ttk` widgets are designed to look more modern and consistent across different operating systems.
- **Usage in Code**: It is used for the more complex parts of the UI, specifically the **Treeview** (the table) and the styled **Buttons**.
- **Styling**: `ttk` allows for the use of the `ttk.Style()` object, which is how the maroon headers and custom button colors are defined in the script.

# A. Book Class

```
class Book:
    def __init__(self, title, author, isbn, is_available=True):
        self._title = title
        self._author = author
        self._isbn = isbn
        self._is_available = is_available

    def get_title(self): return self._title
    def get_author(self): return self._author
    def get_isbn(self): return self._isbn
    def is_available(self): return self._is_available
    def set_available(self, status): self._is_available = status
```

## 1. The Constructor (__init__)

The constructor initializes a new Book object with four specific attributes:

- self._title: Stores the name of the book.
- self._author: Stores the author's name.

- self._isbn: Stores the unique ISBN code used for identification and filtering in the UI.
- self._is_available: A boolean value (True/False) that tracks if the book is currently in the library or checked out. By default, new books are set to True (Available).

Note on Encapsulation: Notice the underscore (_) before each attribute name (e.g., _title). This is a Python convention indicating these are "protected" members that should not be accessed directly from outside the class.

---

## 2. Accessor Methods (Getters)

To retrieve the protected information safely, the class provides "Getter" methods:

- get_title(): Returns the book's title.
- get_author(): Returns the author's name.
- get_isbn(): Returns the ISBN string, which is crucial for the Library class to locate specific records.
- is_available(): Returns the current status of the book.

---

## 3. Mutator Method (Setter)

The class includes one "Setter" method to modify data:

- set_available(status): This allows the program to update a book's availability. For example, when the "CHANGE STATUS" button is clicked in the GUI, this method is called to flip the value between True and False

## B. Library Class

```
class Library:
    def __init__(self):
        self._books = []
```

```
        self.load_sample_books()

    def load_sample_books(self):
        # Fixed syntax errors from source (missing commas)
        sample_books = [
            ("Python Crash Course", "Rommel Robert Florido",
"978-1593279288"),
            ("Clean Code", "Lanz Isaac Olaes", "978-0132350884"),
            ("The Pragmatic Programmer", "Ernesto Lucas Mateo",
"978-0201616224"),
            ("Head First Design Patterns", "Steven Wayne Chua",
"978-0596007126"),
            ("Automate the Boring Stuff", "Pete Andre Ron Borromeo",
"978-1593275990"),
            ("Introduction to Algorithms", "Nadine Endrenal",
"978-0262033848"),
            ("Grokking Algorithms", "Charish Aiwyne De Ocampo",
"978-1617292231"),
            ("Fluent Python", "Sean Kavin Fermo", "978-1492056348"),
            ("Data Algorithms", "Brian Miguel Rabe", "978-1516487521"),
            ("Fluent Python", "Aldryn Untalan", "978-1234567890")
        ]
        for title, author, isbn in sample_books:
            self._books.append(Book(title, author, isbn))

    def get_books(self):
        return self._books

    def add_book(self, book):
        self._books.append(book)

    def toggle_status(self, isbn):
        for book in self._books:
            if book.get_isbn() == isbn:
                book.set_available(not book.is_available())
                return True
        return False
```

The Library class acts as the backend engine for the application. It manages the collection of books and performs all data-related operations without interacting with the UI directly.

---

### A. Composition and Storage

The class initializes with a protected list, self._books = [], which is used to store multiple instances of the Book class.

This is a demonstration of composition, where one class contains others to function.

---

**B. Data Initialization** (load_sample_books): Upon instantiation, the class populates itself with a hardcoded list of sample books, creating new Book objects for each entry.

---

### C. Data Operations:

- add_book(self, book): Accepts a Book object and appends it to the internal database list.
- toggle_status(self, isbn): This is the primary logic method. It iterates through the list to find a book with a matching ISBN. Once found, it uses the Book object's setter method to flip its availability status.

---

## C. Library GUI

```
class LibraryGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Enverga University - Library Manager")
        self.root.state('zoomed')
        self.root.configure(bg="#f4f4f7")

        self.library = Library()
        self.style = ttk.Style()
        self.setup_styles()

        # --- SCROLLABLE CONTAINER ---
        self.main_canvas = tk.Canvas(self.root, bg="#f4f4f7",
highlightthickness=0)
        self.scrollbar = ttk.Scrollbar(self.root, orient="vertical",
command=self.main_canvas.yview)
        self.scrollable_frame = tk.Frame(self.main_canvas, bg="#f4f4f7")

        self.scrollable_frame.bind(
            "<Configure>",
            lambda e:
self.main_canvas.configure(scrollregion=self.main_canvas.bbox("all"))
        )
        self.canvas_window = self.main_canvas.create_window((0, 0),
window=self.scrollable_frame, anchor="nw")
        self.main_canvas.bind('<Configure>', self._on_canvas_configure)

        self.root.bind_all("<MouseWheel>", self._on_mousewheel)
        self.root.bind_all("<Button-4>", self._on_mousewheel)
        self.root.bind_all("<Button-5>", self._on_mousewheel)

        self.main_canvas.configure(yscrollcommand=self.scrollbar.set)
        self.scrollbar.pack(side="right", fill="y")
        self.main_canvas.pack(side="left", fill="both", expand=True)

        self.setup_ui()
```

```
        self.refresh_list()

    def _on_mousewheel(self, event):
        if event.num == 4 or (hasattr(event, 'delta') and event.delta >
0):
            self.main_canvas.yview_scroll(-1, "units")
        elif event.num == 5 or (hasattr(event, 'delta') and event.delta
< 0):
            self.main_canvas.yview_scroll(1, "units")

    def _on_canvas_configure(self, event):
        self.main_canvas.itemconfig(self.canvas_window,
width=event.width)

    def setup_styles(self):
        self.style.theme_use('clam')
        self.style.configure("Treeview", background="white",
fieldbackground="white",
                             rowheight=50, font=("Segoe UI", 11),
borderwidth=0)
        self.style.configure("Treeview.Heading", font=("Segoe UI Bold",
11),
                             background="#800000", foreground="white",
relief="flat")
        self.style.map("Treeview.Heading", background=[('active',
'#660000')])
        self.style.map("Treeview", background=[('selected', '#f2d7d7')],
                       foreground=[('selected', '#800000')])

        self.style.configure("Primary.TButton", font=("Segoe UI Bold",
10), background="#800000", foreground="white", padding=8, width=20)
        self.style.configure("Secondary.TButton", font=("Segoe UI
Semibold", 10), background="#e9ecef", foreground="#333", padding=8,
width=20)
        self.style.configure("Danger.TButton", font=("Segoe UI Bold",
10), background="#fdf2f2", foreground="#c92a2a", padding=8, width=18)

    def setup_ui(self):
        # MAIN HEADER
```

```
        header = tk.Frame(self.scrollable_frame, bg="#800000",
height=130)
        header.pack(fill="x")
        tk.Label(header, text="UNIVERSITY LIBRARY SYSTEM", bg="#800000",
fg="white",
                 font=("Segoe UI Bold", 32)).pack(pady=(30, 5))
        tk.Label(header, text="Library Management Portal", bg="#800000",
fg="#ffcccc",
                 font=("Segoe UI", 12, "italic")).pack(pady=(0, 30))

        content = tk.Frame(self.scrollable_frame, bg="#f4f4f7")
        content.pack(fill="both", expand=True, padx=100, pady=30)

        # --- COMPACT INPUT CARD ---
        input_card = tk.Frame(content, bg="white",
highlightbackground="#ced4da", highlightthickness=1)
        input_card.pack(fill="x", pady=(0, 30))

        # Maroon Header for the Card
        input_header = tk.Frame(input_card, bg="#800000")
        input_header.pack(fill="x")
        tk.Label(input_header, text="ADD NEW RECORD", bg="#800000",
fg="white",
                 font=("Segoe UI Bold", 11)).pack(pady=10) # White text

        # Body of the Card [cite: 15]
        input_body = tk.Frame(input_card, bg="white", padx=40, pady=20)
        input_body.pack(fill="x")
        input_body.grid_columnconfigure((0, 1, 2), weight=1)

        fields = [("Book Title", "title"), ("Author Name", "author"),
("ISBN Code", "isbn")]
        self.entries = {}
        for i, (label_text, key) in enumerate(fields):
            container = tk.Frame(input_body, bg="white")
            container.grid(row=0, column=i, padx=15, pady=5)

            tk.Label(container, text=label_text.upper(), bg="white",
                     font=("Segoe UI Bold", 8),
```

```
fg="#adb5bd").pack(anchor="center")

            entry = ttk.Entry(container, width=30) # Slightly shorter
for compactness [cite: 17]
            entry.pack(pady=(5, 0), ipady=3)
            self.entries[key] = entry

        # Button Container
        btn_container = tk.Frame(input_body, bg="white")
        btn_container.grid(row=1, column=0, columnspan=3, pady=(25, 5),
sticky="ew")

        actions_frame = tk.Frame(btn_container, bg="white")
        actions_frame.pack(side="left")
        ttk.Button(actions_frame, text="ADD TO COLLECTION",
style="Primary.TButton", command=self.add_book).pack(side="left",
padx=(0, 10))
        ttk.Button(actions_frame, text="CHANGE STATUS",
style="Secondary.TButton", command=self.toggle_status).pack(side="left",
padx=10)
        ttk.Button(btn_container, text="WIPE DATABASE",
style="Danger.TButton", command=self.clear_all).pack(side="right")

        # --- TABLE CARD ---
        table_card = tk.Frame(content, bg="white", padx=30, pady=25,
                              highlightbackground="#ced4da",
highlightthickness=1)
        table_card.pack(fill="both", expand=True)

        search_frame = tk.Frame(table_card, bg="white")
        search_frame.pack(fill="x", pady=(0, 20))
        tk.Label(search_frame, text="FILTER BY ISBN:", bg="white",
font=("Segoe UI Bold", 9), fg="#495057").pack(side="left", padx=(0, 10))
        self.search_entry = ttk.Entry(search_frame, width=40)
        self.search_entry.pack(side="left", ipady=2)
        self.search_entry.bind("<KeyRelease>", lambda e:
self.refresh_list())

        columns = ("title", "author", "isbn", "status")
```

```
        self.tree = ttk.Treeview(table_card, columns=columns,
show="headings", height=15)

        for col in columns:
            self.tree.heading(col, text=col.upper())

        # Alignment logic
        self.tree.column("title", width=400, anchor="w")
        self.tree.column("author", width=250, anchor="w")
        self.tree.column("isbn", width=180, anchor="center")
        self.tree.column("status", width=180, anchor="center")

        self.tree.tag_configure('available', foreground='#2f9e44',
font=("Segoe UI Semibold", 11))
        self.tree.tag_configure('checkedout', foreground='#e03131',
font=("Segoe UI Semibold", 11))
        self.tree.tag_configure('even', background='#fcfcfc')

        self.tree.pack(side="left", fill="both", expand=True)
        list_sb = ttk.Scrollbar(table_card, orient="vertical",
command=self.tree.yview)
        self.tree.configure(yscrollcommand=list_sb.set)
        list_sb.pack(side="right", fill="y")

    def refresh_list(self):
        for item in self.tree.get_children():
            self.tree.delete(item)
        query = self.search_entry.get().lower()
        for i, book in enumerate(self.library.get_books()):
            if query in book.get_isbn().lower():
                status = "●  Available" if book.is_available() else "○
Checked Out"
                tag = 'available' if book.is_available() else
'checkedout'
                row_tag = (tag, 'even' if i % 2 == 0 else 'odd')
                self.tree.insert("", "end", values=(book.get_title(),
book.get_author(),
                                                    book.get_isbn(),
status), tags=row_tag)
```

```
    def add_book(self):
        t, a, i = self.entries['title'].get(),
self.entries['author'].get(), self.entries['isbn'].get()
        if t and a and i:
            self.library.add_book(Book(t, a, i))
            for e in self.entries.values():
                e.delete(0, tk.END)
            self.refresh_list()
        else:
            messagebox.showwarning("Incomplete", "Please fill all
fields.")

    def toggle_status(self):
        selected = self.tree.focus()
        if not selected:
            return
        isbn = self.tree.item(selected)['values'][2]
        self.library.toggle_status(isbn)
        self.refresh_list()

    def clear_all(self):
        if messagebox.askyesno("Confirm", "Wipe all library data?"):
            self.library._books.clear()
            self.refresh_list()
```

### The LibraryGUI Class (View & Event Handler)

The LibraryGUI class handles the presentation layer and user interactions. It is responsible for building the window and translating user actions into logic commands for the Library class.

---

**Window and Style Setup**: It initializes the main application window, setting its title to "Enverga University - Library Manager" and configuring a modern "clam" theme with custom styles for buttons and tables.

---

**Complex Layout Management:**

**Scrollable Canvas**: Because library lists can be long, it uses a tk.Canvas and tk.Frame combination to allow vertical scrolling through the entire interface.

**Component Cards:** The UI is divided into distinct "cards" (frames with borders). The Input Card captures new book data, while the Table Card displays the current inventory.

---

**Event Handling and UI Updates:**

- add_book(self): Retrieves user input from the entry fields, creates a new Book instance via the Library class, and clears the inputs.
- refresh_list(self): Clears the current table and repopulates it. It checks each book's status and applies color-coded tags: green (●) for "Available" and red (○) for "Checked Out".
- toggle_status(self): Identifies the selected row in the table, extracts the ISBN, and commands the Library controller to update that specific record.

## III. Source Code

```
import tkinter as tk
from tkinter import messagebox, ttk

class Book:
    def __init__(self, title, author, isbn, is_available=True):
        self._title = title
        self._author = author
        self._isbn = isbn
        self._is_available = is_available

    def get_title(self): return self._title
    def get_author(self): return self._author
    def get_isbn(self): return self._isbn
    def is_available(self): return self._is_available
    def set_available(self, status): self._is_available = status

class Library:
```

```
    def __init__(self):
        self._books = []
        self.load_sample_books()

    def load_sample_books(self):
        # Fixed syntax errors from source (missing commas)
        sample_books = [
            ("Python Crash Course", "Rommel Robert Florido",
"978-1593279288"),
            ("Clean Code", "Lanz Isaac Olaes", "978-0132350884"),
            ("The Pragmatic Programmer", "Ernesto Lucas Mateo",
"978-0201616224"),
            ("Head First Design Patterns", "Steven Wayne Chua",
"978-0596007126"),
            ("Automate the Boring Stuff", "Pete Andre Ron
Borromeo", "978-1593275990"),
            ("Introduction to Algorithms", "Nadine Endrenal",
"978-0262033848"),
            ("Grokking Algorithms", "Charish Aiwyne De Ocampo",
"978-1617292231"),
            ("Fluent Python", "Sean Kavin Fermo",
"978-1492056348"),
            ("Data Algorithms", "Brian Miguel Rabe",
"978-1516487521"),
            ("Fluent Python", "Aldryn Untalan", "978-1234567890")
        ]
        for title, author, isbn in sample_books:
            self._books.append(Book(title, author, isbn))

    def get_books(self):
        return self._books

    def add_book(self, book):
        self._books.append(book)
```

```
    def toggle_status(self, isbn):
        for book in self._books:
            if book.get_isbn() == isbn:
                book.set_available(not book.is_available())
                return True
        return False

class LibraryGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Enverga University - Library Manager")
        self.root.state('zoomed')
        self.root.configure(bg="#f4f4f7")

        self.library = Library()
        self.style = ttk.Style()
        self.setup_styles()

        # --- SCROLLABLE CONTAINER ---
        self.main_canvas = tk.Canvas(self.root, bg="#f4f4f7",
highlightthickness=0)
        self.scrollbar = ttk.Scrollbar(self.root,
orient="vertical", command=self.main_canvas.yview)
        self.scrollable_frame = tk.Frame(self.main_canvas,
bg="#f4f4f7")

        self.scrollable_frame.bind(
            "<Configure>",
            lambda e:
self.main_canvas.configure(scrollregion=self.main_canvas.bbox("all"
))
        )
        self.canvas_window = self.main_canvas.create_window((0, 0),
window=self.scrollable_frame, anchor="nw")
        self.main_canvas.bind('<Configure>',
```

```
self._on_canvas_configure)

        self.root.bind_all("<MouseWheel>", self._on_mousewheel)
        self.root.bind_all("<Button-4>", self._on_mousewheel)
        self.root.bind_all("<Button-5>", self._on_mousewheel)


self.main_canvas.configure(yscrollcommand=self.scrollbar.set)
        self.scrollbar.pack(side="right", fill="y")
        self.main_canvas.pack(side="left", fill="both",
expand=True)

        self.setup_ui()
        self.refresh_list()

    def _on_mousewheel(self, event):
        if event.num == 4 or (hasattr(event, 'delta') and
event.delta > 0):
            self.main_canvas.yview_scroll(-1, "units")
        elif event.num == 5 or (hasattr(event, 'delta') and
event.delta < 0):
            self.main_canvas.yview_scroll(1, "units")

    def _on_canvas_configure(self, event):
        self.main_canvas.itemconfig(self.canvas_window,
width=event.width)

    def setup_styles(self):
        self.style.theme_use('clam')
        self.style.configure("Treeview", background="white",
fieldbackground="white",
                             rowheight=50, font=("Segoe UI", 11),
borderwidth=0)
        self.style.configure("Treeview.Heading", font=("Segoe UI
Bold", 11),
```

```
                                background="#800000",
foreground="white", relief="flat")
        self.style.map("Treeview.Heading", background=[('active',
'#660000')])
        self.style.map("Treeview", background=[('selected',
'#f2d7d7')],
                       foreground=[('selected', '#800000')])

        self.style.configure("Primary.TButton", font=("Segoe UI
Bold", 10), background="#800000", foreground="white", padding=8,
width=20)
        self.style.configure("Secondary.TButton", font=("Segoe UI
Semibold", 10), background="#e9ecef", foreground="#333", padding=8,
width=20)
        self.style.configure("Danger.TButton", font=("Segoe UI
Bold", 10), background="#fdf2f2", foreground="#c92a2a", padding=8,
width=18)

    def setup_ui(self):
        # MAIN HEADER
        header = tk.Frame(self.scrollable_frame, bg="#800000",
height=130)
        header.pack(fill="x")
        tk.Label(header, text="UNIVERSITY LIBRARY SYSTEM",
bg="#800000", fg="white",
                 font=("Segoe UI Bold", 32)).pack(pady=(30, 5))
        tk.Label(header, text="Library Management Portal",
bg="#800000", fg="#ffcccc",
                 font=("Segoe UI", 12, "italic")).pack(pady=(0,
30))

        content = tk.Frame(self.scrollable_frame, bg="#f4f4f7")
        content.pack(fill="both", expand=True, padx=100, pady=30)

        # --- COMPACT INPUT CARD ---
```

```
        input_card = tk.Frame(content, bg="white",
highlightbackground="#ced4da", highlightthickness=1)
        input_card.pack(fill="x", pady=(0, 30))

        # Maroon Header for the Card
        input_header = tk.Frame(input_card, bg="#800000")
        input_header.pack(fill="x")
        tk.Label(input_header, text="ADD NEW RECORD", bg="#800000",
fg="white",
                 font=("Segoe UI Bold", 11)).pack(pady=10) # White
text

        # Body of the Card [cite: 15]
        input_body = tk.Frame(input_card, bg="white", padx=40,
pady=20)
        input_body.pack(fill="x")
        input_body.grid_columnconfigure((0, 1, 2), weight=1)

        fields = [("Book Title", "title"), ("Author Name",
"author"), ("ISBN Code", "isbn")]
        self.entries = {}
        for i, (label_text, key) in enumerate(fields):
            container = tk.Frame(input_body, bg="white")
            container.grid(row=0, column=i, padx=15, pady=5)

            tk.Label(container, text=label_text.upper(),
bg="white",
                     font=("Segoe UI Bold", 8),
fg="#adb5bd").pack(anchor="center")

            entry = ttk.Entry(container, width=30) # Slightly
shorter for compactness [cite: 17]
            entry.pack(pady=(5, 0), ipady=3)
            self.entries[key] = entry
```

```
        # Button Container
        btn_container = tk.Frame(input_body, bg="white")
        btn_container.grid(row=1, column=0, columnspan=3, pady=(25,
5), sticky="ew")

        actions_frame = tk.Frame(btn_container, bg="white")
        actions_frame.pack(side="left")
        ttk.Button(actions_frame, text="ADD TO COLLECTION",
style="Primary.TButton", command=self.add_book).pack(side="left",
padx=(0, 10))
        ttk.Button(actions_frame, text="CHANGE STATUS",
style="Secondary.TButton",
command=self.toggle_status).pack(side="left", padx=10)
        ttk.Button(btn_container, text="WIPE DATABASE",
style="Danger.TButton", command=self.clear_all).pack(side="right")

        # --- TABLE CARD ---
        table_card = tk.Frame(content, bg="white", padx=30,
pady=25,
                              highlightbackground="#ced4da",
highlightthickness=1)
        table_card.pack(fill="both", expand=True)

        search_frame = tk.Frame(table_card, bg="white")
        search_frame.pack(fill="x", pady=(0, 20))
        tk.Label(search_frame, text="FILTER BY ISBN:", bg="white",
font=("Segoe UI Bold", 9), fg="#495057").pack(side="left", padx=(0,
10))
        self.search_entry = ttk.Entry(search_frame, width=40)
        self.search_entry.pack(side="left", ipady=2)
        self.search_entry.bind("<KeyRelease>", lambda e:
self.refresh_list())

        columns = ("title", "author", "isbn", "status")
        self.tree = ttk.Treeview(table_card, columns=columns,
```

```
show="headings", height=15)

        for col in columns:
            self.tree.heading(col, text=col.upper())

        # Alignment logic
        self.tree.column("title", width=400, anchor="w")
        self.tree.column("author", width=250, anchor="w")
        self.tree.column("isbn", width=180, anchor="center")
        self.tree.column("status", width=180, anchor="center")

        self.tree.tag_configure('available', foreground='#2f9e44',
font=("Segoe UI Semibold", 11))
        self.tree.tag_configure('checkedout', foreground='#e03131',
font=("Segoe UI Semibold", 11))
        self.tree.tag_configure('even', background='#fcfcfc')

        self.tree.pack(side="left", fill="both", expand=True)
        list_sb = ttk.Scrollbar(table_card, orient="vertical",
command=self.tree.yview)
        self.tree.configure(yscrollcommand=list_sb.set)
        list_sb.pack(side="right", fill="y")

    def refresh_list(self):
        for item in self.tree.get_children():
            self.tree.delete(item)
        query = self.search_entry.get().lower()
        for i, book in enumerate(self.library.get_books()):
            if query in book.get_isbn().lower():
                status = "●  Available" if book.is_available() else
"○  Checked Out"
                tag = 'available' if book.is_available() else
'checkedout'
                row_tag = (tag, 'even' if i % 2 == 0 else 'odd')
                self.tree.insert("", "end",
```

```
values=(book.get_title(), book.get_author(),
                                                book.get_isbn(),
status), tags=row_tag)

    def add_book(self):
        t, a, i = self.entries['title'].get(),
self.entries['author'].get(), self.entries['isbn'].get()
        if t and a and i:
            self.library.add_book(Book(t, a, i))
            for e in self.entries.values():
                e.delete(0, tk.END)
            self.refresh_list()
        else:
            messagebox.showwarning("Incomplete", "Please fill all
fields.")

    def toggle_status(self):
        selected = self.tree.focus()
        if not selected:
            return
        isbn = self.tree.item(selected)['values'][2]
        self.library.toggle_status(isbn)
        self.refresh_list()

    def clear_all(self):
        if messagebox.askyesno("Confirm", "Wipe all library
data?"):
            self.library._books.clear()
            self.refresh_list()

if __name__ == "__main__":
    root = tk.Tk()
    app = LibraryGUI(root)
    root.mainloop()
```

## IV. Areas for Improvement

While the current version of the University Library System is fully functional and meets all core requirements, there are several technical enhancements that could further professionalize the application. One primary area for improvement is the implementation of While the current version of the University Library System is fully functional and meets all core requirements, there are several technical enhancements that could further professionalize the application. One primary area for improvement is the implementation of **Data Persistence**; currently, any new books added or status changes made are stored only in the computer's volatile memory and are lost once the program is closed. Integrating a local SQLite database or a JSON file system would allow the `Library` class to save and reload the collection across different sessions.

Additionally, the user experience could be refined by adding **Input Validation** to the `add_book` method in the `LibraryGUI` class. This would ensure that ISBN codes follow a specific numerical format and that duplicate entries are prevented, thereby maintaining higher data integrity within the system; currently, any new books added or status changes made are stored only in the computer's volatile memory and are lost once the program is closed. Integrating a local SQLite database or a JSON file system would allow the Library class to save and reload the collection across different sessions. Additionally, the user experience could be refined by adding Input Validation to the add_book method in the LibraryGUI class. This would ensure that ISBN codes follow a specific numerical format and that duplicate entries are prevented, thereby maintaining higher data integrity within the system

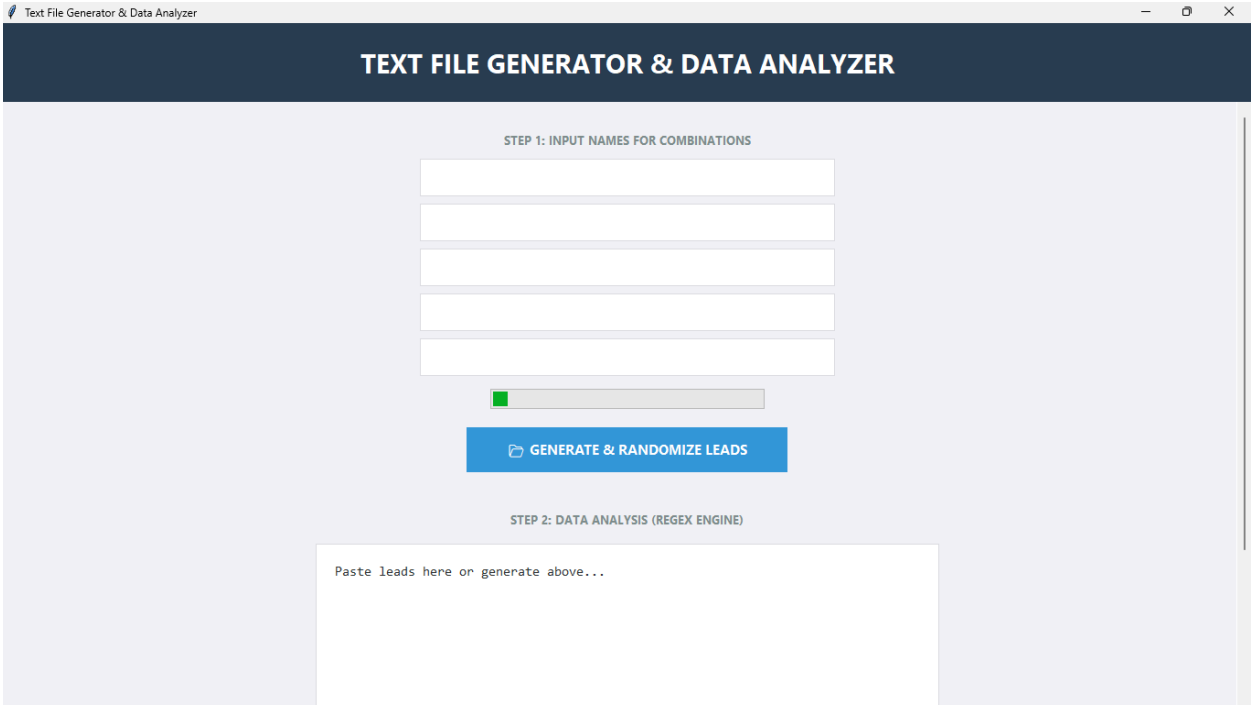
## V. Reflection

Developing this Library Management System provided a valuable opportunity to apply theoretical Object-Oriented Programming (OOP) concepts to a practical, real-world scenario. By structuring the program into distinct Book, Library, and LibraryGUI classes, I gained a deeper understanding of how Composition allows complex systems to be built from simpler, reusable components. Implementing Encapsulation was particularly insightful, as using protected attributes and getter/setter methods demonstrated how to protect internal data from external interference while still providing a controlled interface for the GUI. Overall, this project highlighted the importance of a "Separation of Concerns" architecture, showing how a clean backend logic can seamlessly support a dynamic and interactive frontend.

# Task 2: Object Oriented Programming in Python

## Program : Text File Generator & Data Analyzer

## I. Program Description




The **Text File Generator & Data Analyzer** is a comprehensive Python-based desktop utility designed for efficient lead generation and automated data processing through a professional Tkinter interface. By utilizing combinatorial logic, the program generates unique lead identifiers and email addresses from user-provided name inputs, which are then assigned to various domains using a randomized sampling method to ensure data variety. The application features a streamlined workflow that includes a high-visibility header, a scrollable main container for managing large datasets, and integrated loading indicators to provide real-time feedback during the generation process.

Complementing its generation capabilities, the suite incorporates a robust analysis engine powered by Regular Expressions to scrape and categorize information directly from the built-in text display. This analyzer performs a "Truth Check" on the dataset to calculate the total lead count and determine the dominant domain,

dynamically updating the lead classification to either an academic or general market status based on frequency. To enhance the user experience, the tool also includes practical features such as clipboard integration for quick exports and a clean, light-mode aesthetic that prioritizes visual clarity for detailed data auditing.

## II. Operational Logic

The operational logic of the **Text File Generator & Data Analyzer** is designed around a multi-threaded architecture that seamlessly integrates automated data synthesis with precise analytical scraping. By decoupling the computationally intensive combinatorial generation from the main user interface thread, the application maintains a fluid user experience even when processing hundreds of lead permutations. This logic flow transitions from a randomized data creation stage—where names are paired and assigned to domains—to a rigorous data auditing stage powered by Regular Expressions, allowing for real-time statistical insights and list classification

### A. Libraries Imported

```
import re
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from collections import Counter
import random
import itertools
import threading
import time
```

- **`re`:** Used for Regular Expression matching to scrape emails and Lead IDs from text.
- **`tkinter` & `ttk`:** The core framework for the GUI and the themed progress bar.

- **`collections.Counter`:** Efficiently counts domain occurrences to determine the dominant list type.
- **`itertools.permutations`:** Generates all possible unique pairs from the name input list.
- **`threading` & `time`:** Allows the generation process to run in the background so the UI stays responsive.

### B. LeadApp Class

```
class LeadApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Text File Generator & Data Analyzer")
        self.root.state('zoomed')

        # LIGHT MODE COLORS
        self.bg_main = "#f0f2f5"      # Light grey background
        self.bg_card = "#ffffff"      # Pure white for containers
        self.fg_text = "#2c3e50"      # Dark slate for text
        self.border_col = "#dcdde1"   # Light border
        self.blue = "#3498db"         # Soft blue accent
        self.green = "#2ecc71"        # Emerald green accent

        self.root.configure(bg=self.bg_main)

        # --- HEADER ---
        header = tk.Frame(root, bg="#2c3e50", pady=20)
        header.pack(fill="x")
        tk.Label(header, text="TEXT FILE GENERATOR & DATA ANALYZER",
                 font=('Segoe UI Bold', 22), bg="#2c3e50",
fg="white").pack()
```

```
        self.setup_scrollable_frames()
        self.setup_generator_ui()
        self.setup_auditor_ui()

    def setup_scrollable_frames(self):
        self.canvas = tk.Canvas(self.root, bg=self.bg_main,
highlightthickness=0)
        self.scrollbar = tk.Scrollbar(self.root, orient="vertical",
command=self.canvas.yview)
        self.scrollable_frame = tk.Frame(self.canvas, bg=self.bg_main)

        self.scrollable_frame.bind("<Configure>", lambda e:
self.canvas.configure(scrollregion=self.canvas.bbox("all")))
        self.canvas.create_window((0, 0), window=self.scrollable_frame,
anchor="nw", width=self.root.winfo_screenwidth())
        self.canvas.configure(yscrollcommand=self.scrollbar.set)

        self.scrollbar.pack(side="right", fill="y")
        self.canvas.pack(side="left", fill="both", expand=True)

    def setup_generator_ui(self):
        tk.Label(self.scrollable_frame, text="STEP 1: INPUT NAMES FOR
COMBINATIONS",
                 font=("Segoe UI Bold", 10), bg=self.bg_main,
fg="#7f8c8d").pack(pady=(30, 5))

        # ENTRY BOXES WITH LIGHT THEME
        self.name_entries = []
        for i in range(5):
            entry = tk.Entry(self.scrollable_frame, width=50,
font=("Segoe UI", 12), bd=0,
                             bg=self.bg_card, fg=self.fg_text,
insertbackground="black",
                             highlightthickness=1,
highlightbackground=self.border_col, justify="center")
            entry.pack(pady=4, ipady=8)
            self.name_entries.append(entry)

        self.progress = ttk.Progressbar(self.scrollable_frame,
```

```
mode='indeterminate', length=300)
        self.progress.pack(pady=10)

        tk.Button(self.scrollable_frame, text="📂 GENERATE & RANDOMIZE
LEADS", command=self.start_gen_thread,
                  bg=self.blue, fg="white", font=("Segoe UI Bold", 11),
padx=40, pady=10, bd=0, cursor="hand2").pack(pady=10)

    def setup_auditor_ui(self):
        tk.Label(self.scrollable_frame, text="STEP 2: DATA ANALYSIS
(REGEX ENGINE)",
                 font=("Segoe UI Bold", 10), bg=self.bg_main,
fg="#7f8c8d").pack(pady=(30, 5))

        # TEXT BOX (PLACEHOLDERS)
        self.text_display = tk.Text(self.scrollable_frame, height=10,
width=80, font=('Consolas', 11),
                                    bg=self.bg_card, fg="#2d3436", bd=0,
padx=20, pady=20,
                                    highlightthickness=1,
highlightbackground=self.border_col)
        self.text_display.pack(pady=10)
        self.text_display.insert("1.0", "Paste leads here or generate
above...")

        # BUTTONS
        btn_frame = tk.Frame(self.scrollable_frame, bg=self.bg_main)
        btn_frame.pack(pady=10)

        tk.Button(btn_frame, text="🚀 RUN REGEX ANALYSIS",
command=self.analyze_emails,
                  bg=self.green, fg="white", font=("Segoe UI Bold", 11),
padx=30, pady=10, bd=0, cursor="hand2").pack(side="left", padx=10)

        tk.Button(btn_frame, text="📋 COPY TO CLIPBOARD",
command=self.copy_to_clipboard,
                  bg="#e67e22", fg="white", font=("Segoe UI Bold", 11),
padx=30, pady=10, bd=0, cursor="hand2").pack(side="left", padx=10)
```

```
        self.res_label = tk.Label(self.scrollable_frame, text="Awaiting
Data...",
                                  font=('Segoe UI Semilight', 13),
bg=self.bg_main, fg=self.fg_text)
        self.res_label.pack(pady=(10, 50))

    def start_gen_thread(self):
        self.progress.start(10)
        threading.Thread(target=self.generate_file, daemon=True).start()

    def generate_file(self):
        all_words = []
        for n in self.name_entries:
            val = n.get().strip()
            if val: all_words.extend(val.split())

        if len(all_words) < 2:
            self.root.after(0, lambda: [self.progress.stop(),
messagebox.showwarning("Error", "Need more names!")])
            return

        time.sleep(0.5)
        domains = ["gmail.com", "outlook.com", "mseuf.edu.ph",
"techcorp.io"]
        name_combinations = list(itertools.permutations(all_words, 2))

        data = [f"Lead ID {random.randint(100, 999)}: {p[0]} {p[1]}
({p[0].lower()}.{p[1].lower()}@{random.choice(domains)})"
                for p in name_combinations]

        self.root.after(0, self.finish_generation, data)

    def finish_generation(self, data):
        self.progress.stop()
        self.text_display.delete("1.0", tk.END)
        self.text_display.insert(tk.END, "\n".join(data))
        messagebox.showinfo("Success", f"Generated {len(data)} unique
combinations.")
```

```
    def copy_to_clipboard(self):
        content = self.text_display.get("1.0", tk.END).strip()
        self.root.clipboard_clear()
        self.root.clipboard_append(content)
        messagebox.showinfo("Clipboard", "Data copied successfully!")

    def analyze_emails(self):
        content = self.text_display.get("1.0", tk.END).strip()
        emails =
re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", content)
        ids = re.findall(r"Lead ID \d+", content) #

        if not emails: return

        counts = Counter([e.split('@')[1] for e in emails])
        top_domain, freq = counts.most_common(1)[0]

        # CLASSIFICATION LOGIC
        status = "MSEUF (Academic List)" if top_domain == "mseuf.edu.ph"
else f"{top_domain.split('.')[0].title()} (General Market List)"

        report = (
            f"📊 TOTAL LEAD ID COUNTER: {len(ids)}\n"
            f"🏢 TOP DOMAIN: {top_domain} ({freq} hits)\n"
            f"🎯 LEAD CLASSIFICATION: {status}"
        )
        self.res_label.config(text=report)
```

The LeadApp class serves as the central orchestration layer of the software, responsible for managing the lifecycle of the graphical user interface and the underlying data processing logic. It acts as a bridge between user inputs and the backend generation and analysis engines.

## Core Responsibilities of the LeadApp Class

- **GUI Initialization and Theming:** Upon instantiation, the class sets up the primary `Tkinter` window, defining the visual identity of the application through a specific "Light Mode" color palette, including background, card, and accent colors.
- **Hierarchical UI Construction**: The class organizes the interface into modular sections by calling `setup_scrollable_frames`, `setup_generator_ui`, and `setup_auditor_ui`, which allows for a clean separation between the input, loading, and results areas.
- **State Managemen**t: It maintains the state of various UI components, such as the list of `name_entries` for input and the `text_display` for output, ensuring data persists as the user moves between generation and analysis.

## Functional Logic within LeadApp

C. **Concurrency Handling:** To maintain a responsive interface, the class utilizes threading via the `start_gen_thread` method. This prevents the UI from freezing while the `generate_file` function performs combinatorial math and artificial delays.

D. **Data Transformation:** The class handles the conversion of raw string inputs into structured lead data. It manages the logic for `itertools.permutations` to create unique name pairs and uses `random.choice` to assign domains

.

E. **Analytical Intelligence**: Through the `analyze_emails` method, the class performs post-processing on the generated text. It leverages Regular Expressions to extract metadata (emails and IDs) and uses the `collections.Counter` to determine the "Lead Classification" based on domain frequency.

F. **User Feedback and Utilities:** The class manages all interactive feedback, including the `messagebox` alerts for success or errors and the `clipboard_append` functionality for data exporting.

## III. Source Code

```
import re
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from collections import Counter
import random
import itertools
import threading
import time

class LeadApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Text File Generator & Data Analyzer")
        self.root.state('zoomed')

        # LIGHT MODE COLORS
        self.bg_main = "#f0f2f5"      # Light grey background
        self.bg_card = "#ffffff"      # Pure white for containers
        self.fg_text = "#2c3e50"      # Dark slate for text
        self.border_col = "#dcdde1"   # Light border
        self.blue = "#3498db"         # Soft blue accent
        self.green = "#2ecc71"        # Emerald green accent

        self.root.configure(bg=self.bg_main)

        # --- HEADER ---
        header = tk.Frame(root, bg="#2c3e50", pady=20)
        header.pack(fill="x")
        tk.Label(header, text="TEXT FILE GENERATOR & DATA ANALYZER",
                 font=('Segoe UI Bold', 22), bg="#2c3e50",
fg="white").pack()

        self.setup_scrollable_frames()
        self.setup_generator_ui()
        self.setup_auditor_ui()
```

```
    def setup_scrollable_frames(self):
        self.canvas = tk.Canvas(self.root, bg=self.bg_main,
highlightthickness=0)
        self.scrollbar = tk.Scrollbar(self.root, orient="vertical",
command=self.canvas.yview)
        self.scrollable_frame = tk.Frame(self.canvas, bg=self.bg_main)

        self.scrollable_frame.bind("<Configure>", lambda e:
self.canvas.configure(scrollregion=self.canvas.bbox("all")))
        self.canvas.create_window((0, 0), window=self.scrollable_frame,
anchor="nw", width=self.root.winfo_screenwidth())
        self.canvas.configure(yscrollcommand=self.scrollbar.set)

        self.scrollbar.pack(side="right", fill="y")
        self.canvas.pack(side="left", fill="both", expand=True)

    def setup_generator_ui(self):
        tk.Label(self.scrollable_frame, text="STEP 1: INPUT NAMES FOR
COMBINATIONS",
                 font=("Segoe UI Bold", 10), bg=self.bg_main,
fg="#7f8c8d").pack(pady=(30, 5))

        # ENTRY BOXES WITH LIGHT THEME
        self.name_entries = []
        for i in range(5):
            entry = tk.Entry(self.scrollable_frame, width=50,
font=("Segoe UI", 12), bd=0,
                             bg=self.bg_card, fg=self.fg_text,
insertbackground="black",
                             highlightthickness=1,
highlightbackground=self.border_col, justify="center")
            entry.pack(pady=4, ipady=8)
            self.name_entries.append(entry)

        self.progress = ttk.Progressbar(self.scrollable_frame,
mode='indeterminate', length=300)
        self.progress.pack(pady=10)

        tk.Button(self.scrollable_frame, text="📂 GENERATE & RANDOMIZE
```

```
LEADS", command=self.start_gen_thread,
                  bg=self.blue, fg="white", font=("Segoe UI Bold", 11),
padx=40, pady=10, bd=0, cursor="hand2").pack(pady=10)

    def setup_auditor_ui(self):
        tk.Label(self.scrollable_frame, text="STEP 2: DATA ANALYSIS
(REGEX ENGINE)",
                 font=("Segoe UI Bold", 10), bg=self.bg_main,
fg="#7f8c8d").pack(pady=(30, 5))

        # TEXT BOX (PLACEHOLDERS)
        self.text_display = tk.Text(self.scrollable_frame, height=10,
width=80, font=('Consolas', 11),
                                    bg=self.bg_card, fg="#2d3436", bd=0,
padx=20, pady=20,
                                    highlightthickness=1,
highlightbackground=self.border_col)
        self.text_display.pack(pady=10)
        self.text_display.insert("1.0", "Paste leads here or generate
above...")

        # BUTTONS
        btn_frame = tk.Frame(self.scrollable_frame, bg=self.bg_main)
        btn_frame.pack(pady=10)

        tk.Button(btn_frame, text="🚀 RUN REGEX ANALYSIS",
command=self.analyze_emails,
                  bg=self.green, fg="white", font=("Segoe UI Bold", 11),
padx=30, pady=10, bd=0, cursor="hand2").pack(side="left", padx=10)

        tk.Button(btn_frame, text="📋 COPY TO CLIPBOARD",
command=self.copy_to_clipboard,
                  bg="#e67e22", fg="white", font=("Segoe UI Bold", 11),
padx=30, pady=10, bd=0, cursor="hand2").pack(side="left", padx=10)

        self.res_label = tk.Label(self.scrollable_frame, text="Awaiting
Data...",
                                  font=('Segoe UI Semilight', 13),
bg=self.bg_main, fg=self.fg_text)
```

```
        self.res_label.pack(pady=(10, 50))

    def start_gen_thread(self):
        self.progress.start(10)
        threading.Thread(target=self.generate_file, daemon=True).start()

    def generate_file(self):
        all_words = []
        for n in self.name_entries:
            val = n.get().strip()
            if val: all_words.extend(val.split())

        if len(all_words) < 2:
            self.root.after(0, lambda: [self.progress.stop(),
messagebox.showwarning("Error", "Need more names!")])
            return

        time.sleep(0.5)
        domains = ["gmail.com", "outlook.com", "mseuf.edu.ph",
"techcorp.io"]
        name_combinations = list(itertools.permutations(all_words, 2))

        data = [f"Lead ID {random.randint(100, 999)}: {p[0]} {p[1]}
({p[0].lower()}.{p[1].lower()}@{random.choice(domains)})"
                for p in name_combinations]

        self.root.after(0, self.finish_generation, data)

    def finish_generation(self, data):
        self.progress.stop()
        self.text_display.delete("1.0", tk.END)
        self.text_display.insert(tk.END, "\n".join(data))
        messagebox.showinfo("Success", f"Generated {len(data)} unique
combinations.")

    def copy_to_clipboard(self):
        content = self.text_display.get("1.0", tk.END).strip()
        self.root.clipboard_clear()
        self.root.clipboard_append(content)
```

```
        messagebox.showinfo("Clipboard", "Data copied successfully!")

    def analyze_emails(self):
        content = self.text_display.get("1.0", tk.END).strip()
        emails =
re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", content)
        ids = re.findall(r"Lead ID \d+", content) #

        if not emails: return

        counts = Counter([e.split('@')[1] for e in emails])
        top_domain, freq = counts.most_common(1)[0]

        # CLASSIFICATION LOGIC
        status = "MSEUF (Academic List)" if top_domain == "mseuf.edu.ph"
else f"{top_domain.split('.')[0].title()} (General Market List)"

        report = (
            f"📊 TOTAL LEAD ID COUNTER: {len(ids)}\n"
            f"🏢 TOP DOMAIN: {top_domain} ({freq} hits)\n"
            f"🎯 LEAD CLASSIFICATION: {status}"
        )
        self.res_label.config(text=report)

if __name__ == "__main__":
    root = tk.Tk()
    app = LeadApp(root)
    root.mainloop()
```

## IV. Areas for Improvement

One significant area for improvement involves enhancing the software's data validation and error-handling capabilities. While the current version effectively alerts users when fewer than two names are provided, it lacks specialized checks for invalid characters or purely numerical inputs that might disrupt the formatting of generated emails. Implementing a more robust sanitation layer would ensure that the combinatorial

engine only processes valid strings, thereby preventing the creation of malformed leads that could compromise the integrity of the data analysis phase.

The performance of the **Data Analyzer** could be further optimized by introducing more granular RegEx patterns and statistical metrics. Currently, the system identifies top domains and total lead counts , but it could be expanded to detect common formatting errors, duplicate entries, or specific anomalies within the text display. Refining these patterns would allow the tool to provide deeper insights, such as identifying potential "bottlenecks" in lead distribution or flagging low-quality handle names that fall below a certain character threshold.

From a user interface perspective, the application could benefit from more advanced interactive features to streamline the workflow between generation and auditing. Integrating a "Clear All" function or the suggested "Tooltips" system would make the interface more intuitive for first-time users. Additionally, while the current scrollable canvas handles content overflow well, implementing a more dynamic search-and-filter system within the results area would allow users to isolate specific domains or IDs without needing to export the data to an external text editor.

## V. Reflection

Developing the **Text File Generator & Data Analyzer** provided a valuable lesson in the importance of threading and asynchronous processing in desktop applications. By offloading the lead generation logic to a background thread, the program successfully avoids UI freezing, which is a common pitfall in Python GUI development. This design choice, paired with the visual feedback of a progress bar, highlights how technical backend decisions directly translate into a more professional and reliable user experience.

The project also reinforced the power of combining combinatorial mathematics with Regular Expressions to solve data-driven tasks. Using `itertools.permutations` to generate unique pairings ensured that the dataset was comprehensive , while the RegEx-based analyzer proved that structured information can be efficiently extracted from unstructured text displays. This synergy between generation and scraping demonstrates a complete data lifecycle within a single class-based architecture, showcasing how Python's standard libraries can be leveraged to build sophisticated automation tools.

Finally, the transition from a rigid "Cartesian Product" logic to a "Randomized Sampling" approach was a critical turning point in the project’s development. This change ensured that the **Lead Classification** results were not predetermined, forcing the logic to actually analyze domain frequencies to determine if a list was an "MSEUF (Academic List)" or a "General Market List". This shift from static to dynamic analysis significantly improved the "intelligence" of the auditor, proving that a program is most effective when its results are derived from the data it processes rather than hardcoded assumptions.